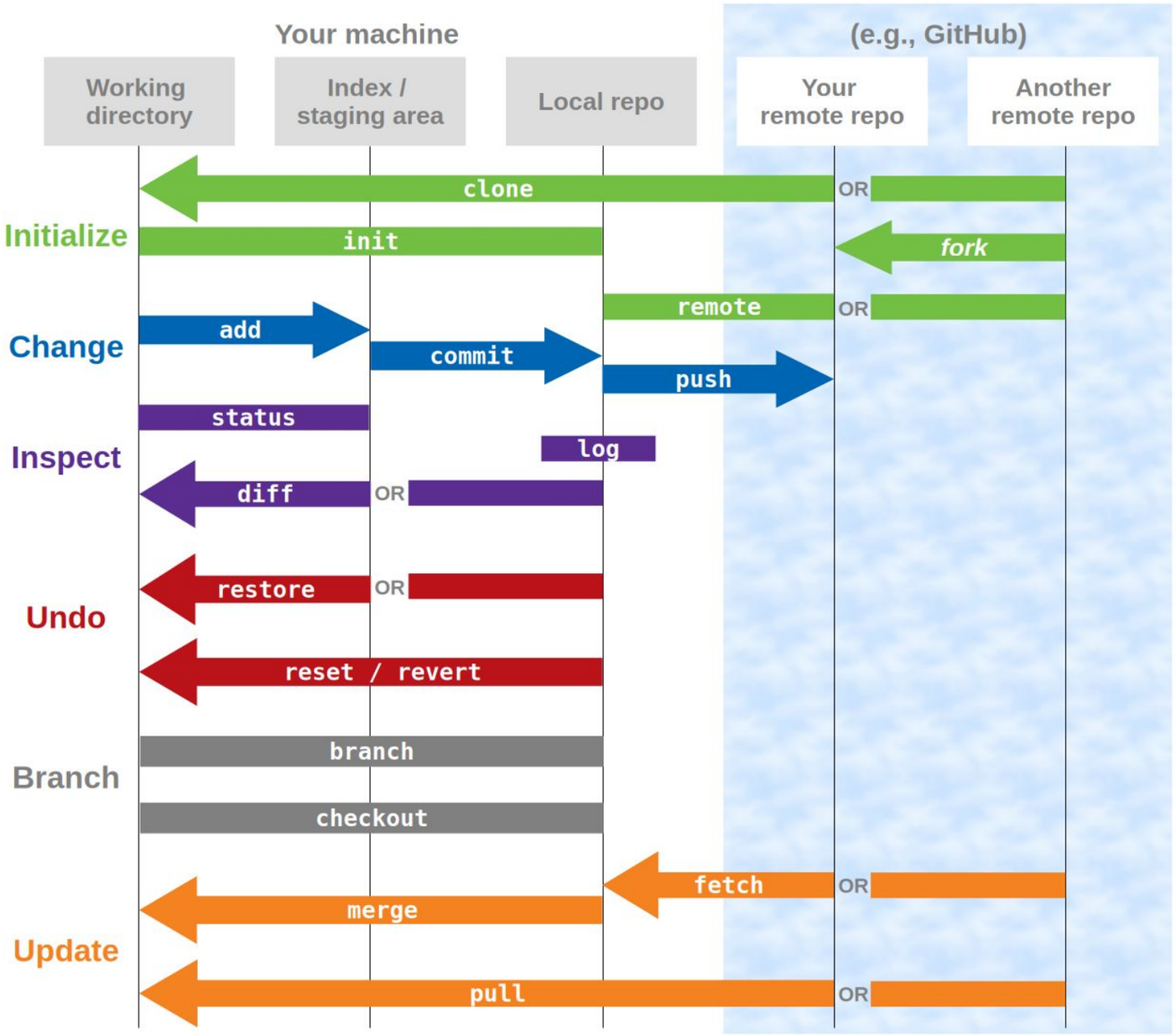
Introduction to Git and GitHub
Part 2

Kendra Oudyk (she/her)

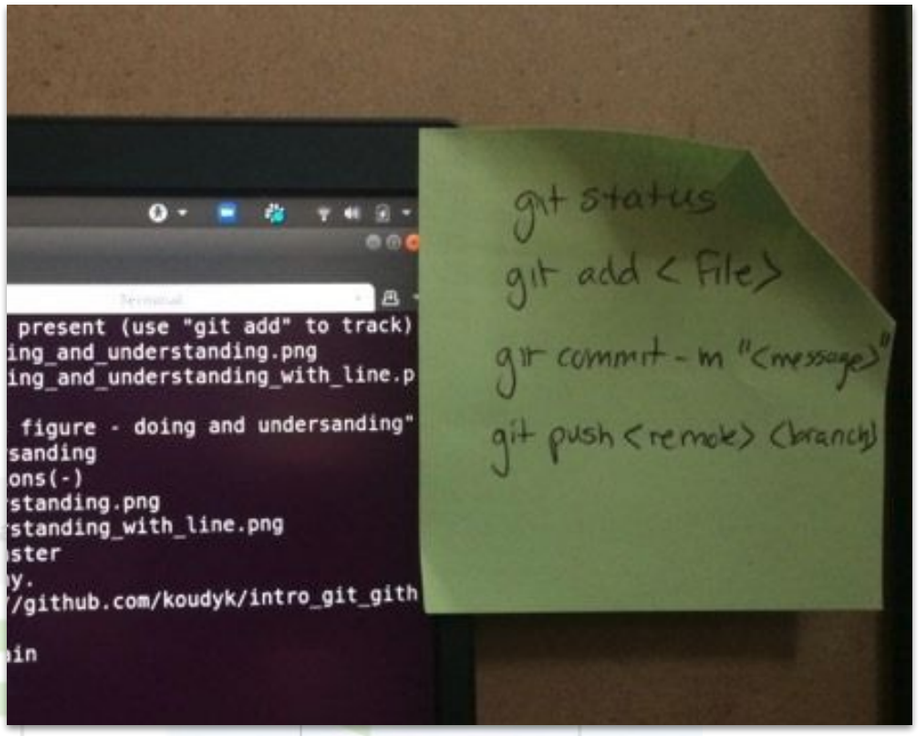
Many parts of this presentation are inspired / based on these great resources

- Chacon, S., & Straub, B. (2014). *Pro git.* Springer Nature. Available at https://git-scm.com/book/en/v2
- The Carpentries. (2021). *Version Control with Git.* https://swcarpentry.github.io/git-novice/.

The commands we'll have covered **by the end of this lecture**



The commands **you might want to remember**



(Everything else you can look up when you need it)

Review: Tracking your work

Put these steps in order:

A) `git commit`

B) Change a file in your working tree

C) `git add`

Review: Tracking and sharing your work

Put these steps in order:

A) Take the snapshot
`git commit`

B) Change a file in your working tree

C) Get the altered file ready to be in the snapshot
`git add`

Goals

Part 1

- What is distributed version control?
- Why is Git useful?
- Track your own work with Git

Part 2

- **Share your work on GitHub (simple / linear collaboration)**
- Collaborate through GitHub (complex / nonlinear collaboration)
- Navigating GitHub
- Try it out!

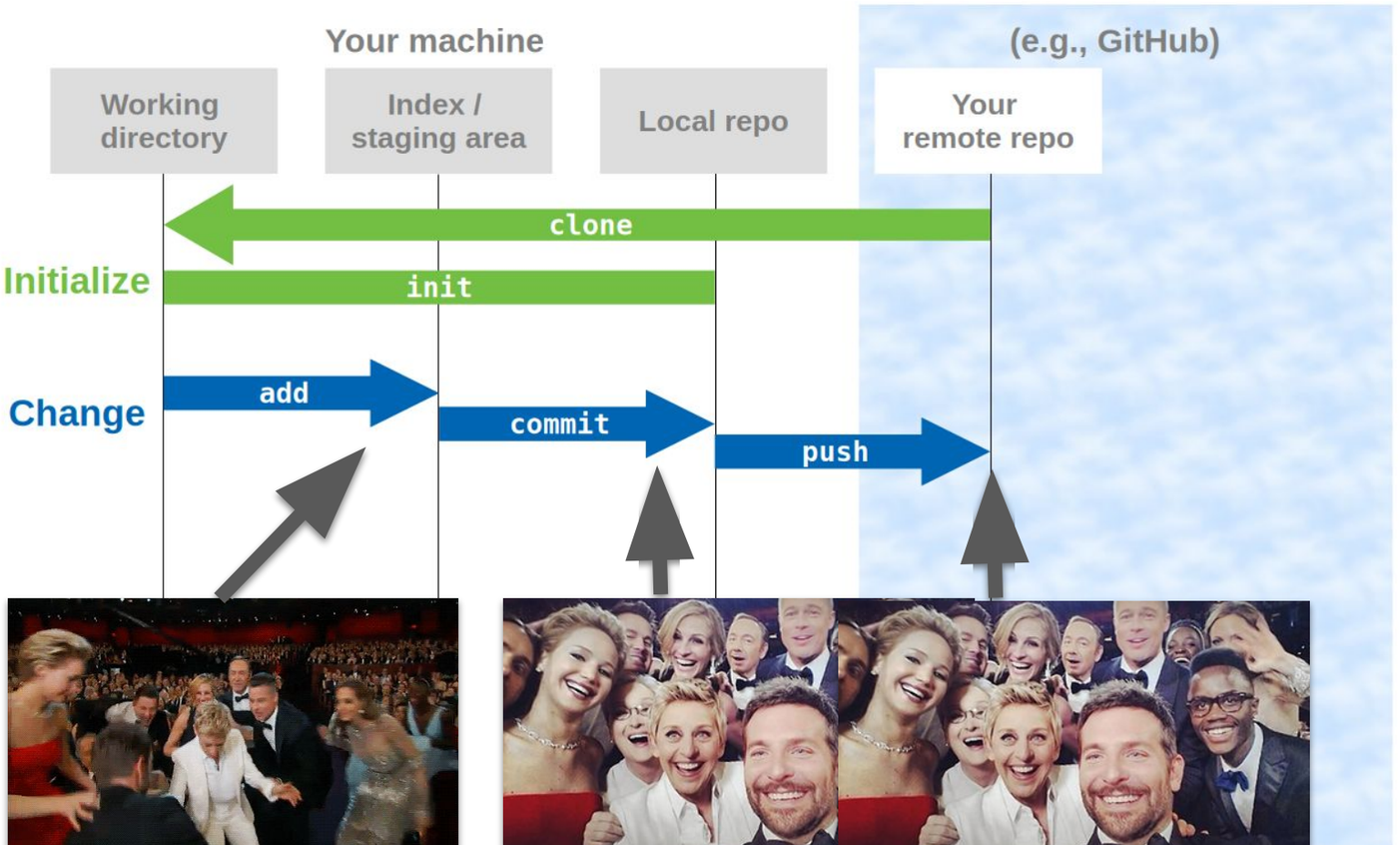
4-step basic workflow for tracking your work **and sharing it**

1. **Modify**
   Change a file in your working tree

2. **Stage**
   `git add <filename>`

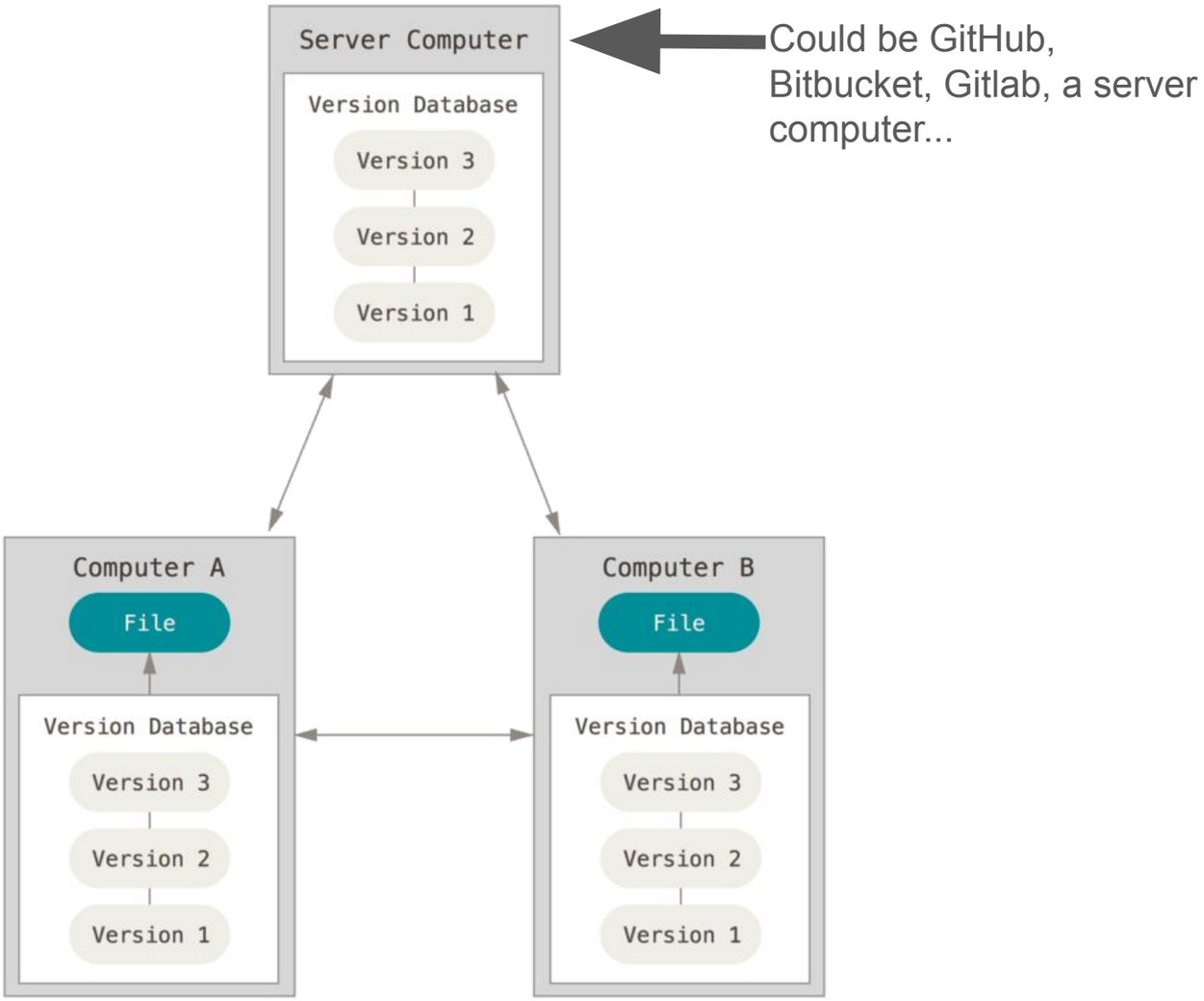
3. **Commit**
   `git commit -m "<short, informative commit message>"`

4. **Push**
   `git push <remote> <branch>`

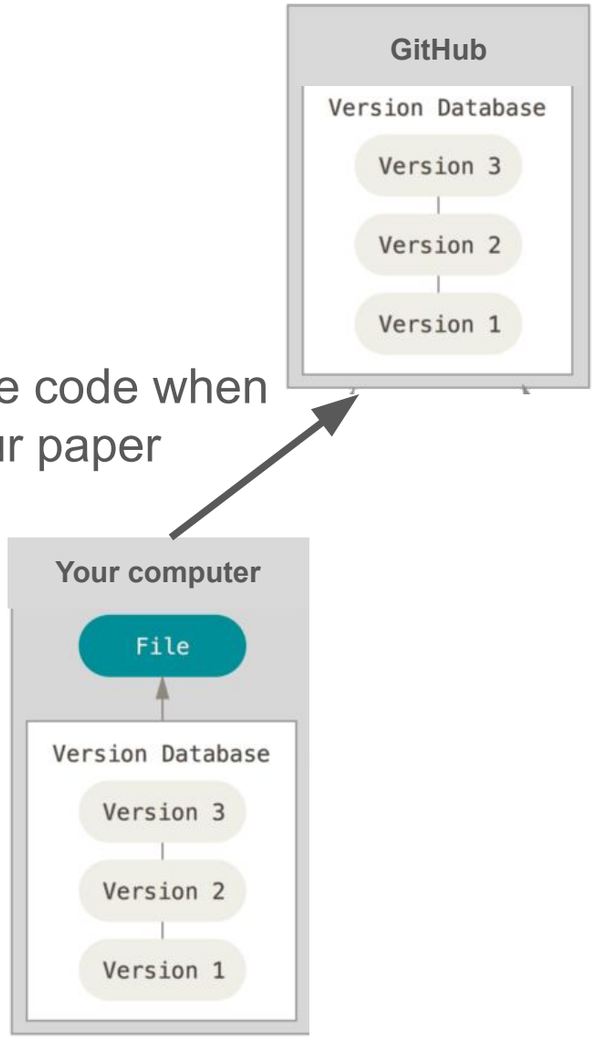


Git vs. GitHub

- Git is the "**language**" we use to do version control.
- GitHub **hosts** git repositories online.



Sharing your work



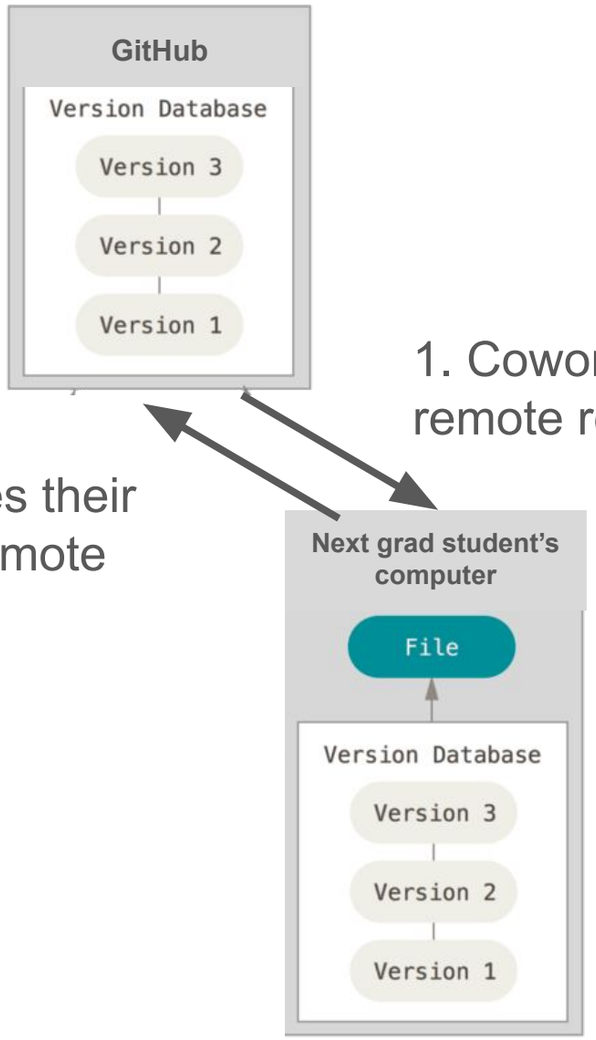
3. You share the code when you publish your paper

1. You write code to run an analysis for your paper

2. You track your work locally

Sharing your work

The next grad student in your lab needs to use your code



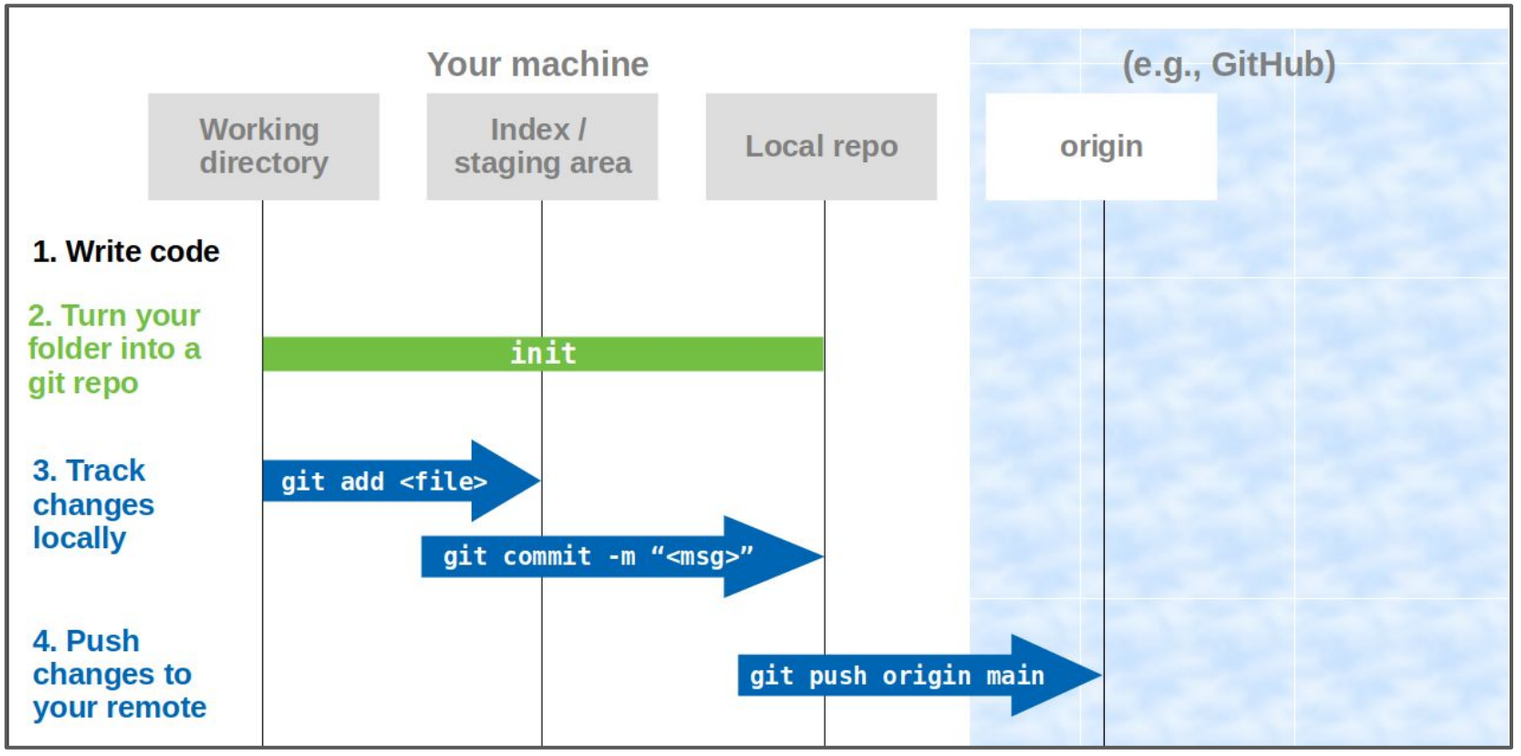
1. Coworker clones your remote repo

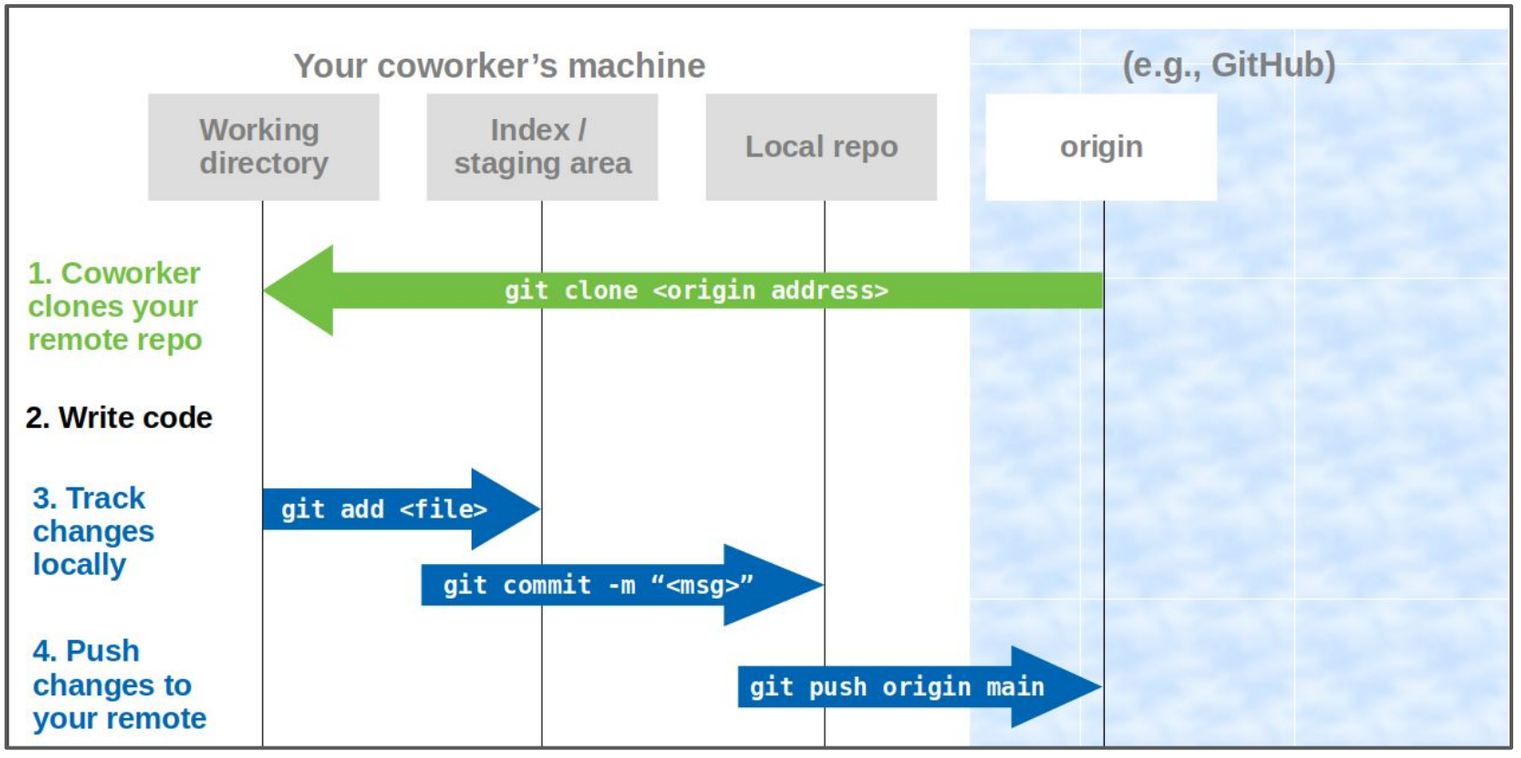
2. Coworker writes code

3. Coworker tracks changes locally

4. Coworker pushes their changes to your remote

Sharing your work





Git commands for sharing your work

- Show your remote repos
  `git remote -v`

- Push commits to a remote repo
  `git push <remote name> <branch>`

Review: Tracking and sharing your work

Put these steps in order:

A) `git add`

B) `git push`

C) `git commit`

D) Change a file in your working tree

Review: Tracking and sharing your work

Put these steps in order:

A) Get the altered file ready to be in the snapshot
`git add`

B) Put the snapshot on GitHub
`git push`

C) Take the snapshot
`git commit`

D) Change a file in your working tree

Goals

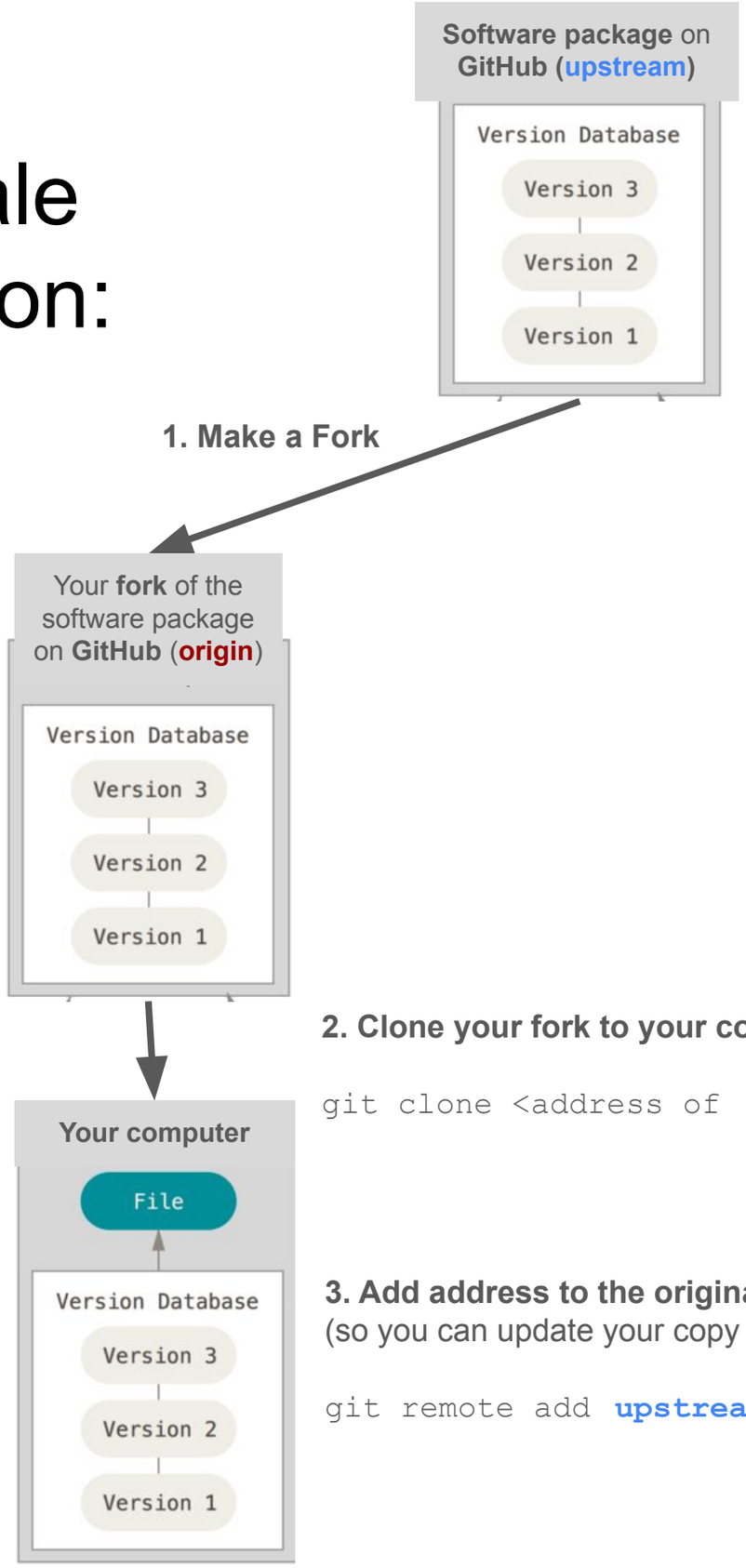
Part 1

- What is distributed version control?
- Why is Git useful?
- Track your own work with Git

Part 2

- Share your work on GitHub (simple / linear collaboration)
- **Collaborate through GitHub (complex / nonlinear collaboration)**
- Navigating GitHub
- Try it out!

Larger-scale collaboration: **Setup**



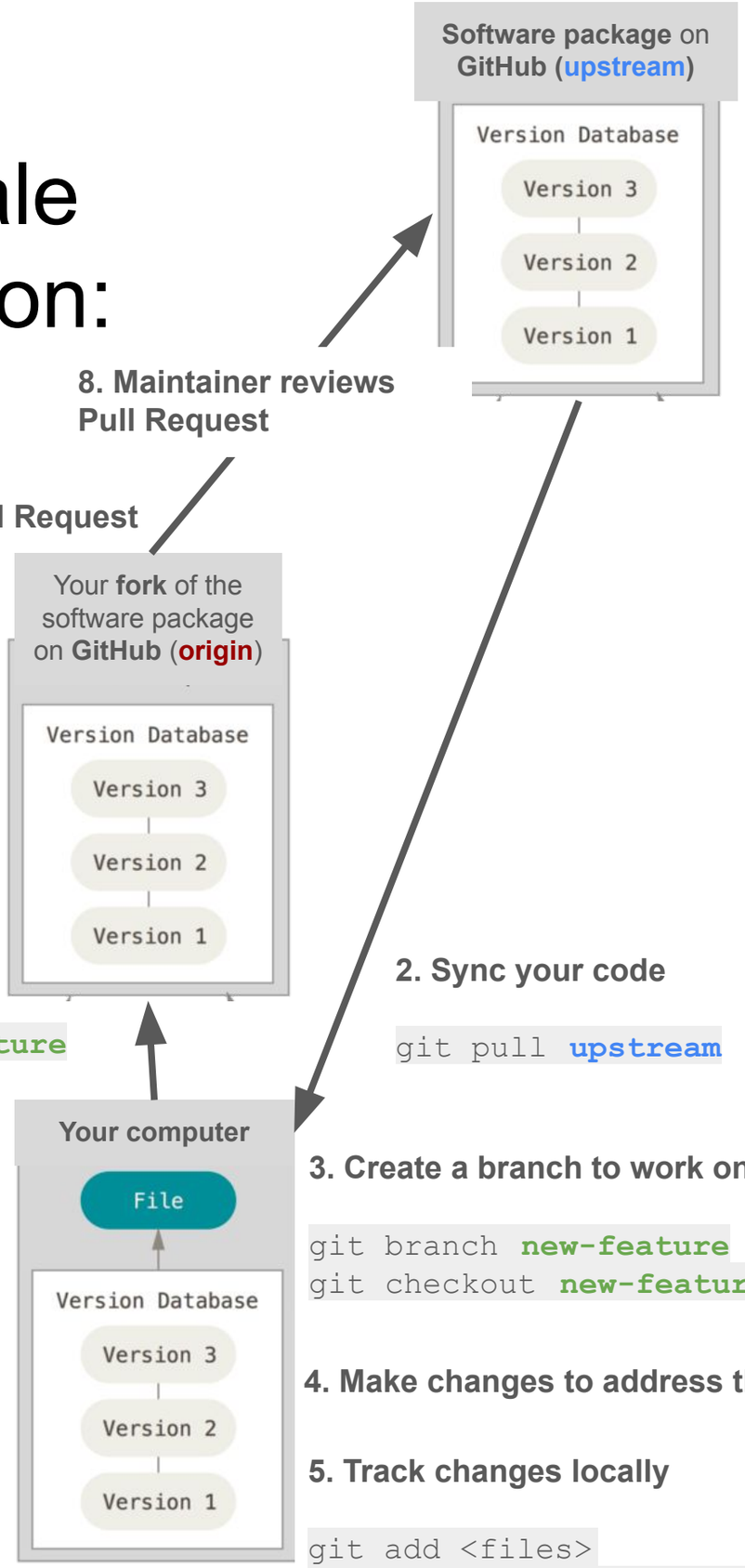
2. Clone your fork to your computer

```
git clone <address of origin>
```

3. Add address to the original repo
(so you can update your copy with others' changes)

```
git remote add upstream <address of upstream>
```

Larger-scale collaboration: **Workflow**



1. Find/make an Issue to work on

2. Sync your code

```
git pull upstream
```

3. Create a branch to work on

```
git branch new-feature
git checkout new-feature
```

4. Make changes to address the issue

5. Track changes locally

```
git add <files>
git commit -m "<informative message>  #<issue number>
```

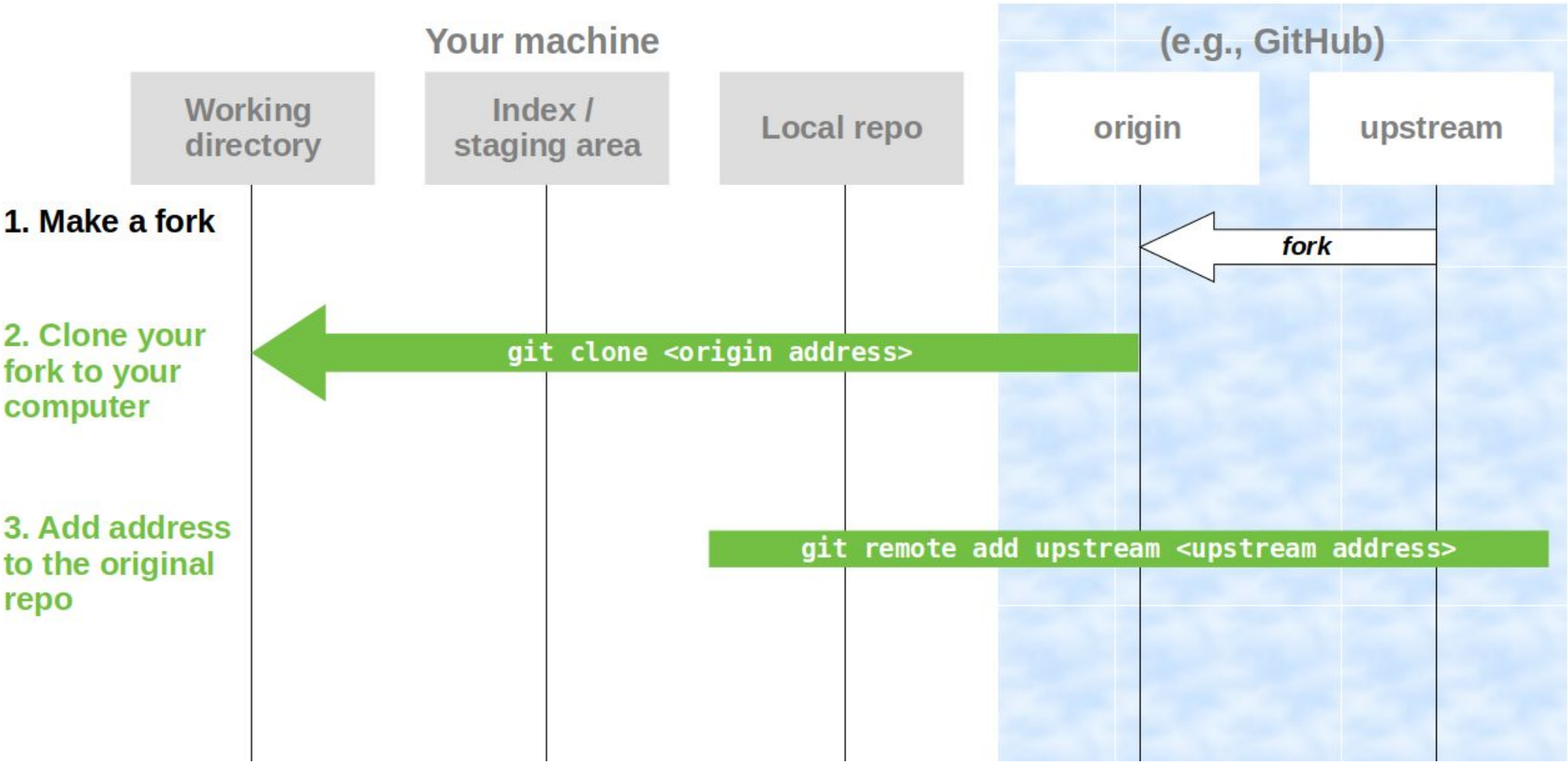
6. Push changes to your fork

```
git push origin new-feature
```

7. You open a Pull Request

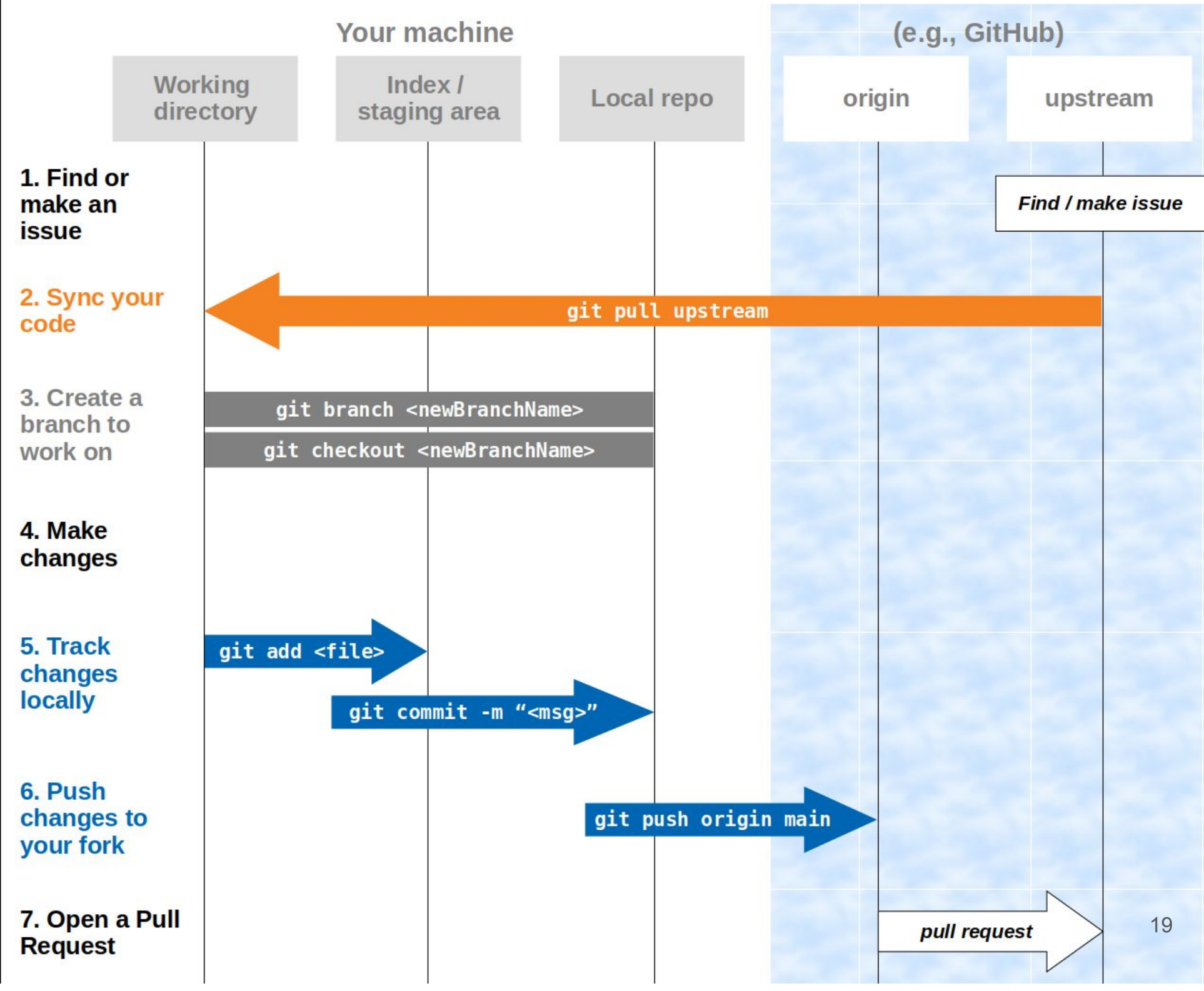
8. Maintainer reviews Pull Request

Larger-scale collaboration: **Setup**

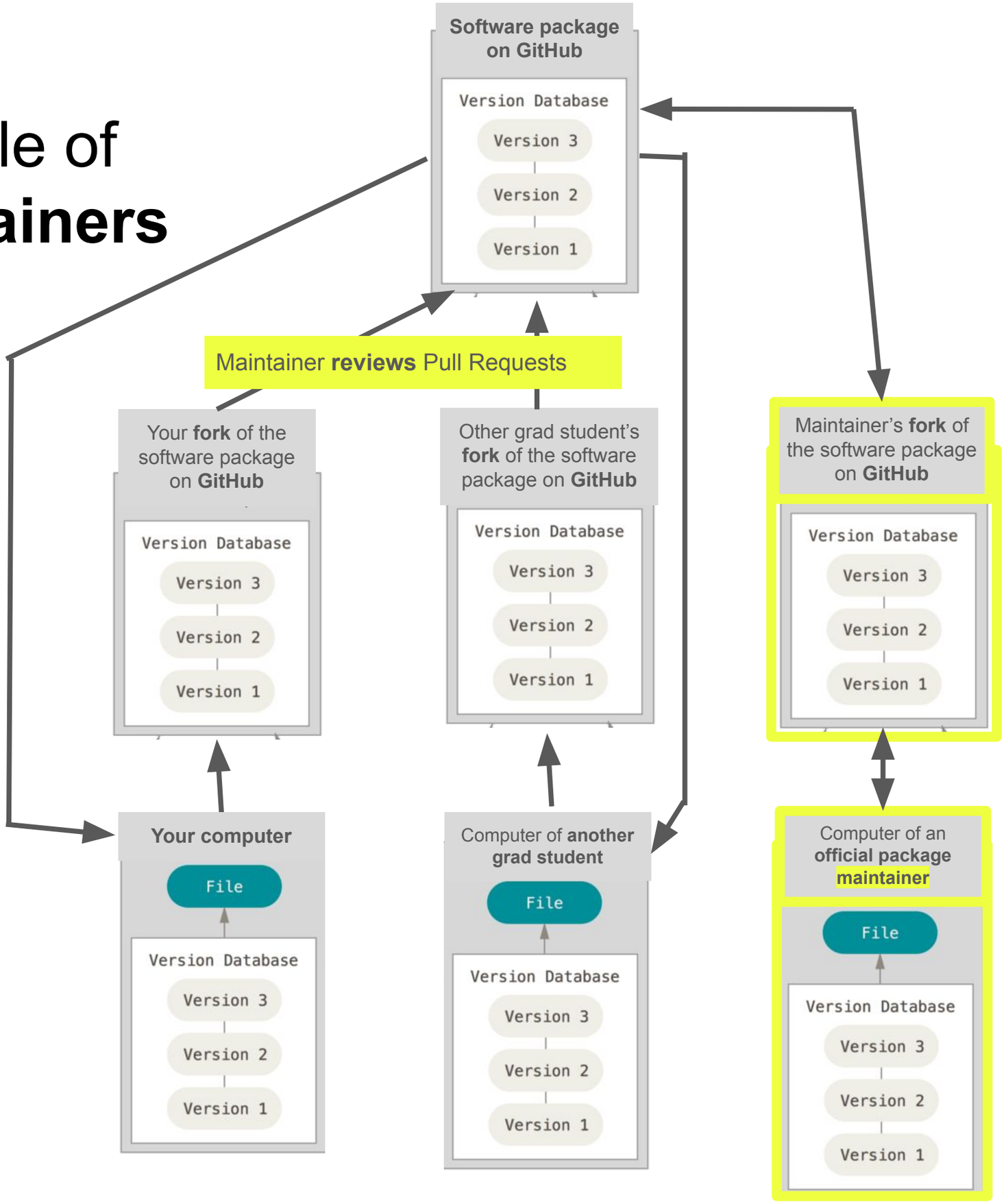


Another way to visualize this

Larger-scale collaboration: **Workflow**



The role of **maintainers**



Git commands for collaboration

- Show your remote repos
  `git remote -v`

- Add a remote repo
  `git remote add <remote name> <remote address>`

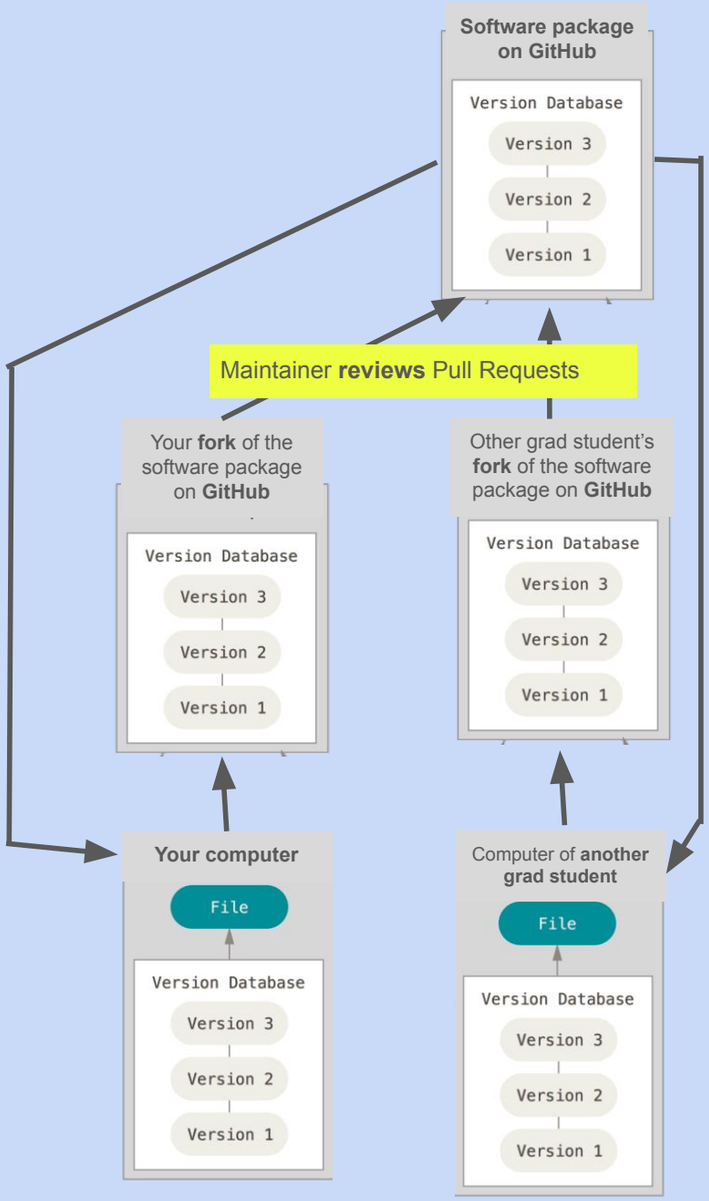
- Push commits to a remote repo
  `git push <remote name> <branch>`

- Pull (fetch and merge) commits from a remote repo
  `git pull <remote name> <branch>`

What do you want to do?

A) **Share your work on GitHub**
(simple / linear collaboration)

B) **Collaborate through GitHub**
(complex / nonlinear collaboration)



Review: Collaborating on GitHub

In a large-scale collaboration workflow, if you want to start working on a new issue, which 2 steps should you do before making any changes to the code?

A) `git pull`

B) `git branch <nameOfNewBranch>`
`git checkout <nameOfNewBranch>`

C) Open a pull request

Review: Collaborating on GitHub

In a large-scale collaboration workflow, if you want to start working on a new issue, what should you do first, before making any changes to the code?

A) Sync your code with your collaborators' changes

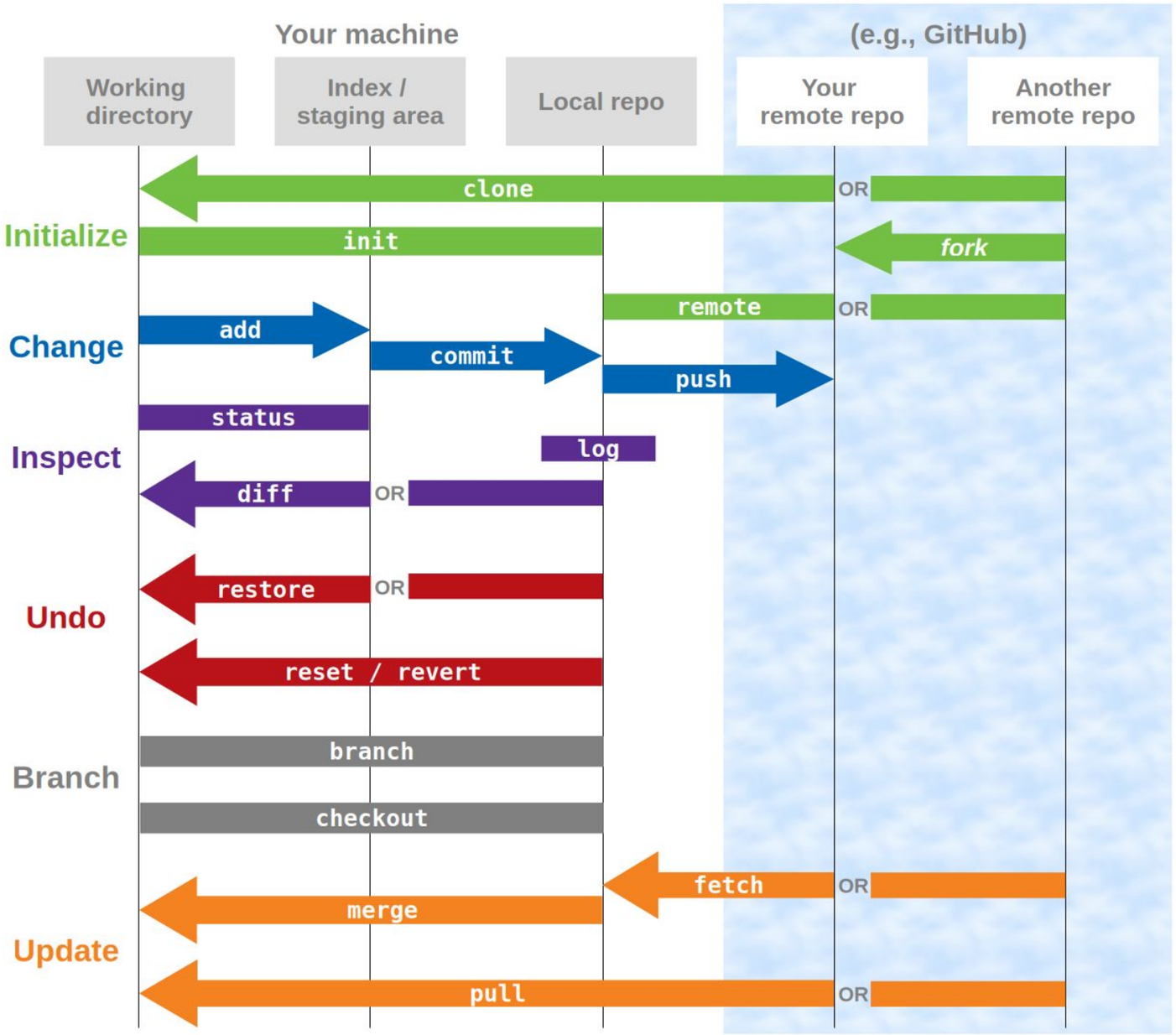
```
git pull
```

B) Start a new branch

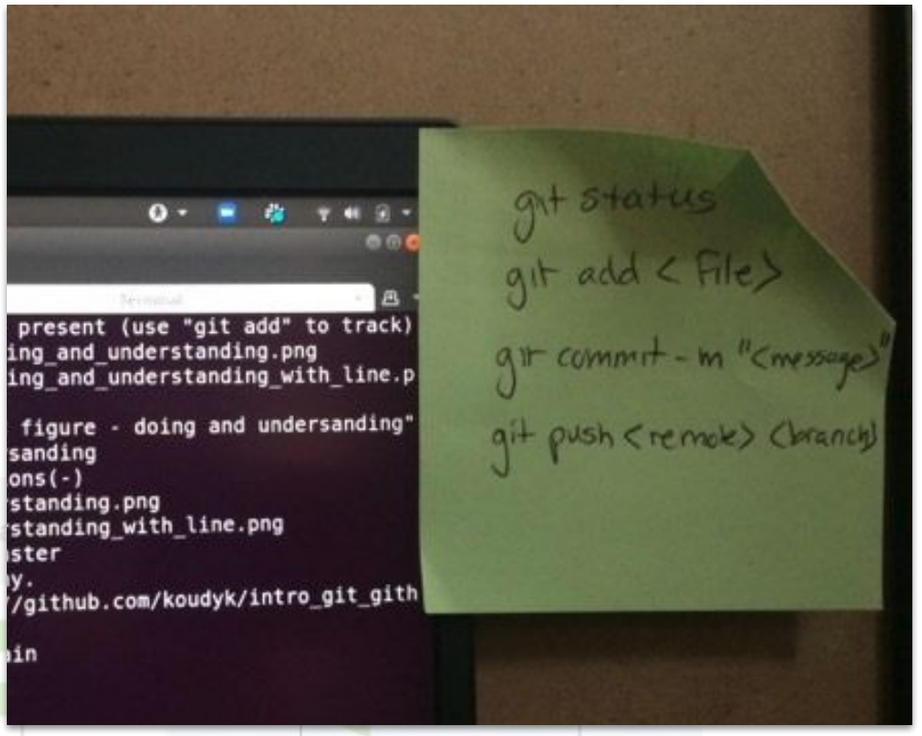
```
git branch <nameOfNewBranch>
git checkout <nameOfNewBranch>
```

C) Open a pull request

The commands we covered



The commands **you might want to remember**



(Everything else you can look up when you need it)

Goals

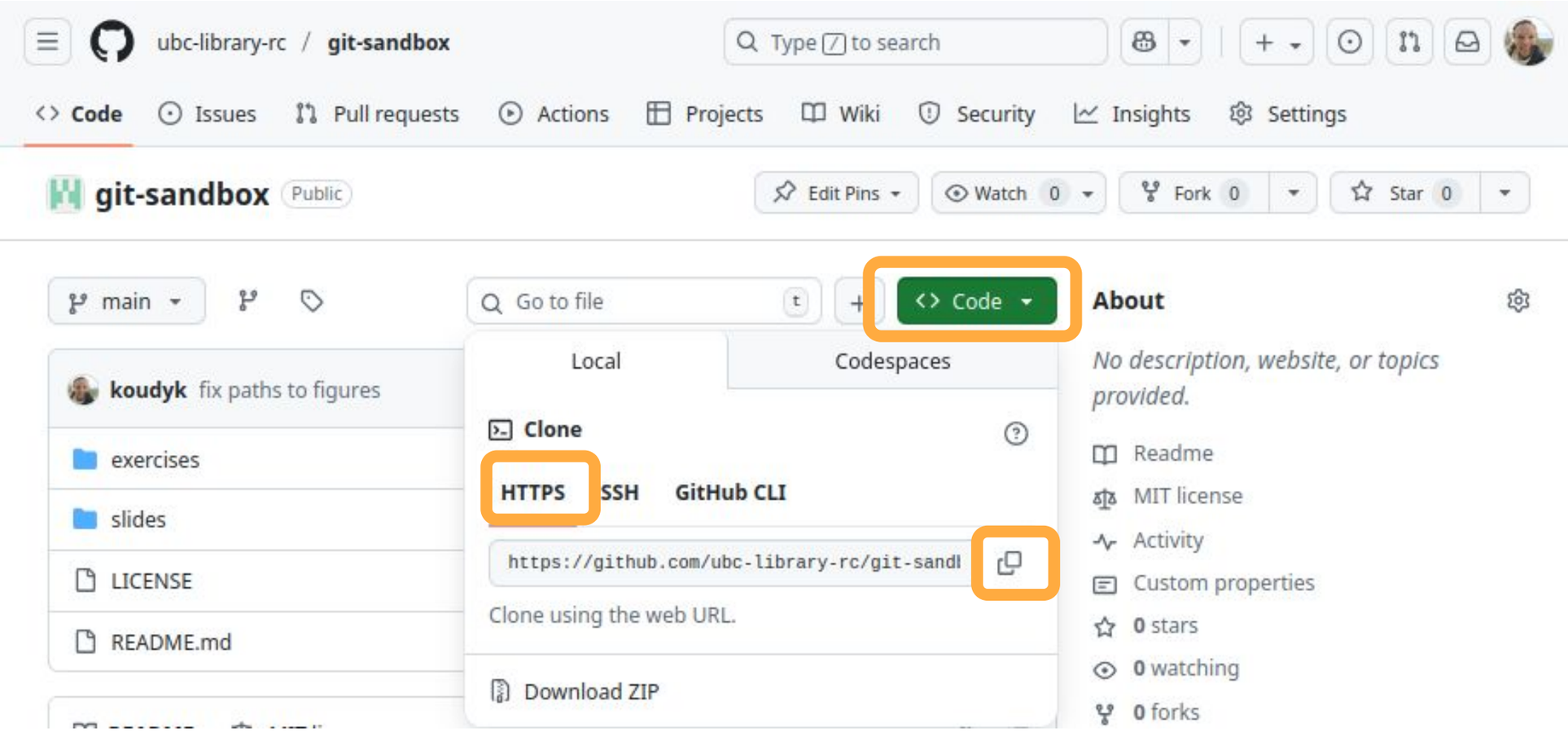
Part 1

- What is distributed version control?
- Why is Git useful?
- Track your own work with Git

Part 2

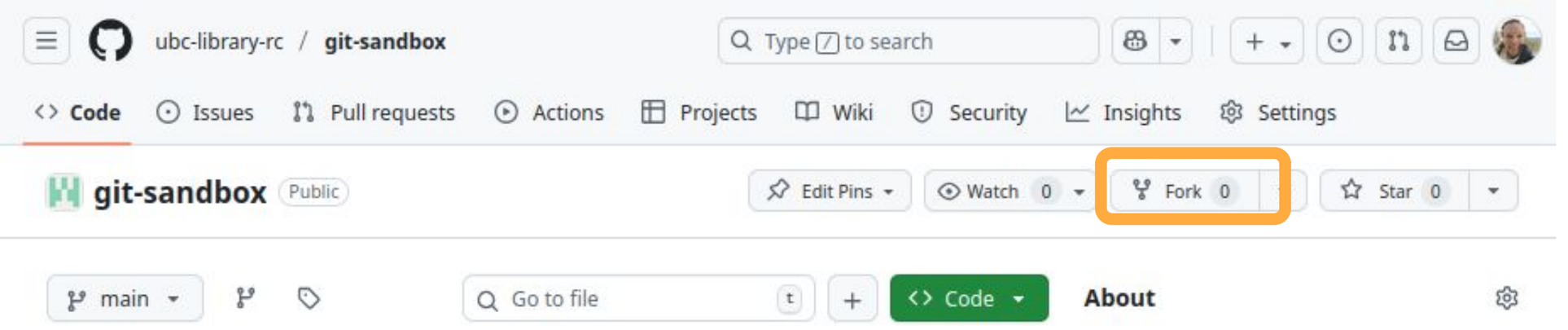
- Share your work on GitHub (simple / linear collaboration)
- Collaborate through GitHub (complex / nonlinear collaboration)
- **Navigating GitHub**
- Try it out!

Navigating GitHub



Where to find address for cloning a repo to your computer

Navigating GitHub



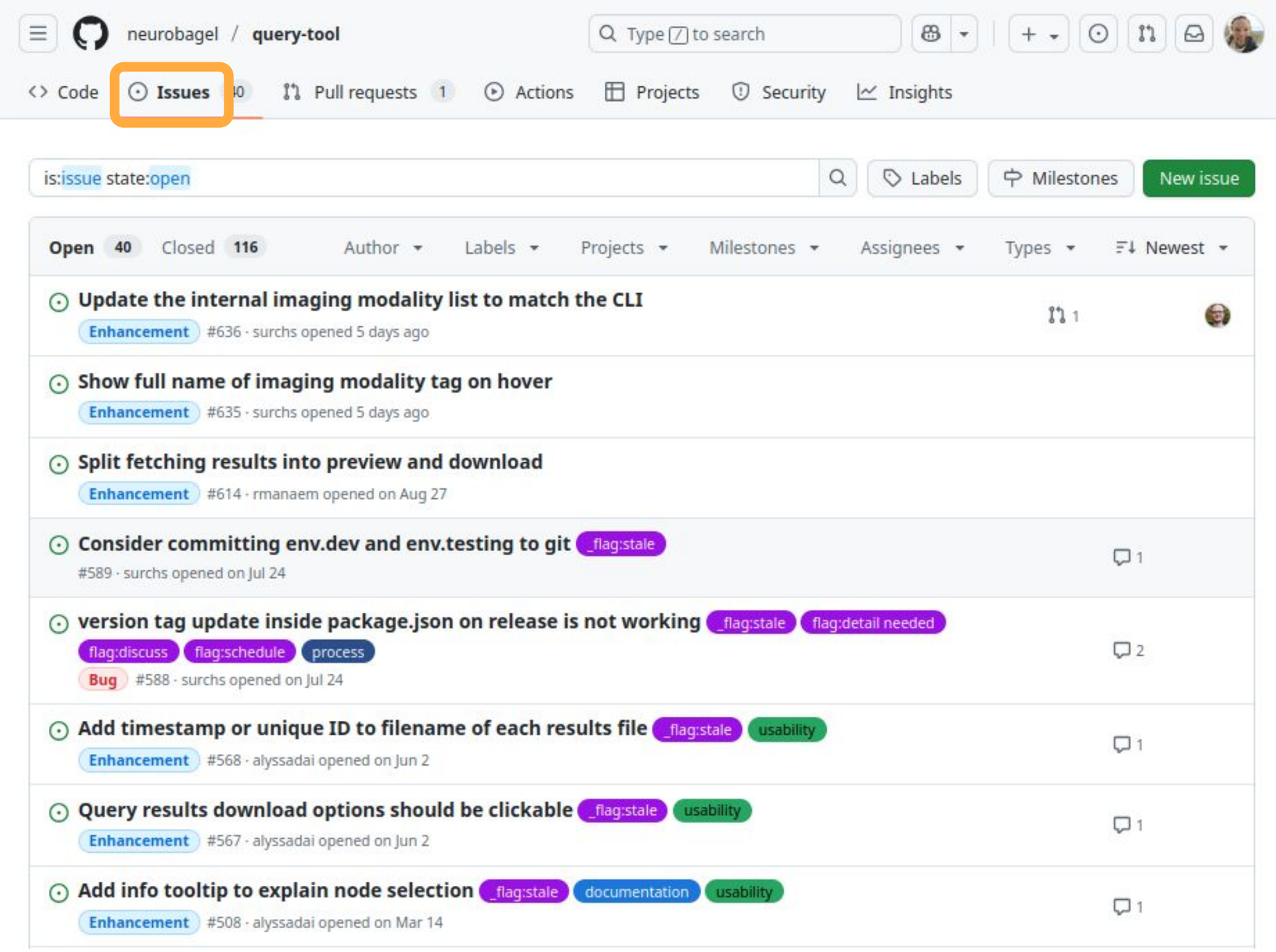
Forks

- Like a clone, but on GitHub
- Most collaborators will have their own fork where they work

Navigating GitHub

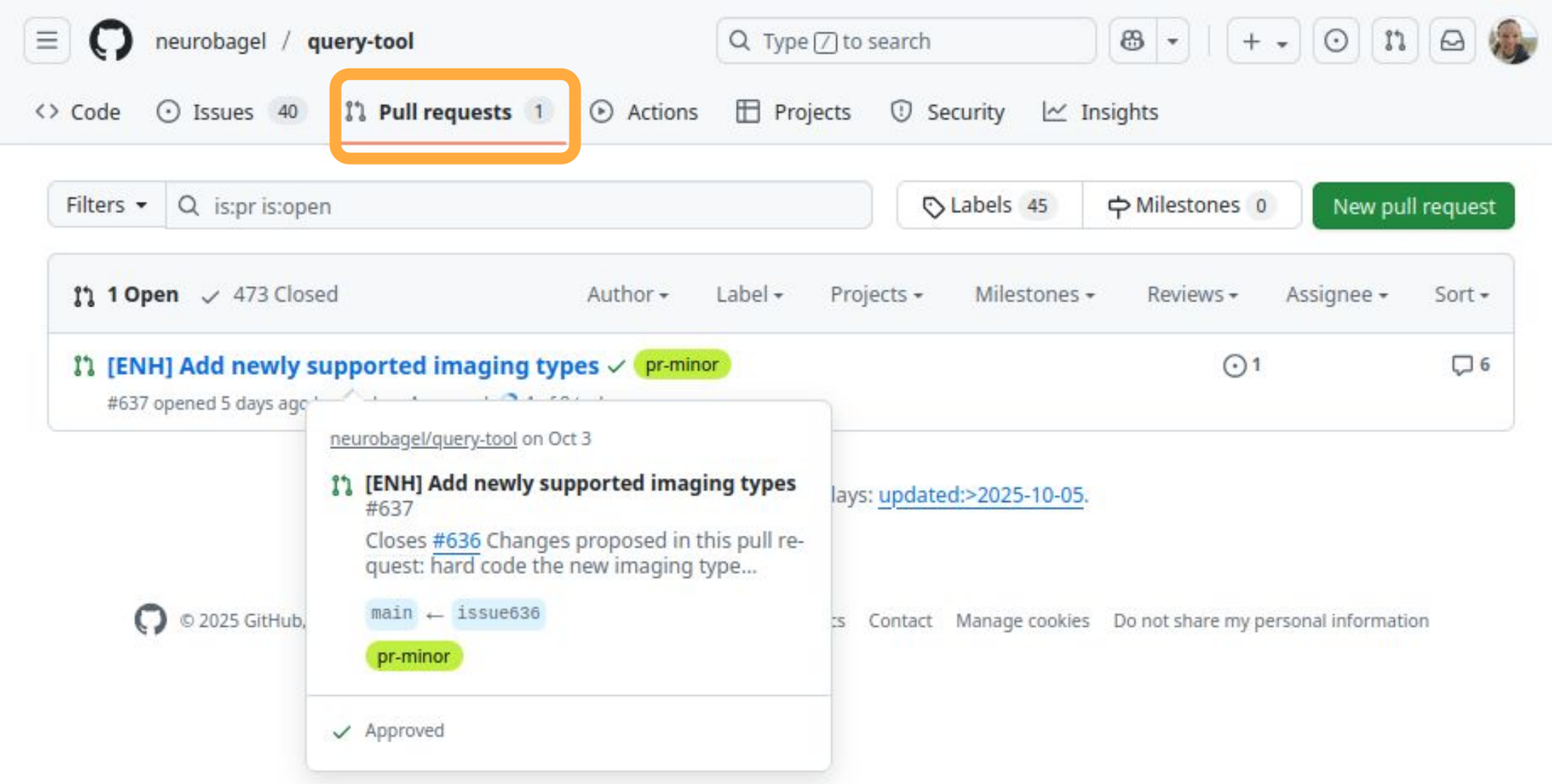
Issues - where you can

- Report a problem
- Propose something new
- Find something to work on



Navigating GitHub

https://github.com/neurobagel/query-tool/pull/637



Pull requests

- Push commits from your own brainch to your own fork, then open a pull request
- Then the project maintainers can
  - review your code
  - make suggestions / edits
  - decide whether to merge it into the original repository

Navigating GitHub

https://github.com/neurobagel/query-tool/pull/637

neurobagel / query-tool

Type / to search

Code | Issues 40 | Pull requests 1 | Actions | Projects | Security | Insights

[ENH] Add newly supported imaging types #637

Code

Open **surchs** wants to merge 2 commits into `main` from `issue636`

Conversation 6 | Commits 2 | Checks 14 | Files changed 1 | +14 −2

surchs commented 5 days ago • edited by alyssadai — Contributor

- Closes Update the internal imaging modality list to match the CLI #636

Changes proposed in this pull request:

- hard code the new imaging types we support, see also Update BIDS imaging suffixes we support bagel-cli#496

Checklist

This section is for the PR reviewer

- [x] PR has an interpretable title with a prefix (`[ENH]` , `[FIX]` , `[REF]` , `[TST]` , `[CI]` , `[MNT]` , `[INF]` , `[MODEL]` , `[DOC]`) *(see our Contributing Guidelines for more info)*
- [ ] PR has a label for the release changelog or `skip-release` (to be applied by maintainers only)
- [x] PR links to GitHub issue with mention `Closes #XXXX`
- [x] Tests pass
- [x] Checks pass
- [ ] If the PR changes the participant-level and/or the dataset-level result file, the `query-tool-results` files in the neurobagel_examples repo have been regenerated

Reviewers

sourcery-ai[bot]

alyssadai

Still in progress? Learn about draft PRs

Assignees

No one assigned

Labels

pr-minor

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close

Goals

Part 1

- What is distributed version control?
- Why is Git useful?
- Track your own work with Git

Part 2

- Share your work on GitHub (simple / linear collaboration)
- Collaborate through GitHub (complex / nonlinear collaboration)
- Navigating GitHub
- **Try it out!**

Check if you're ready

☑️ A) Can you open a bash shell?

- Open a terminal, type `echo $SHELL` and press ENTER.
- The output should include `bash` or `zsh`

☑️ B) Do you have git installed?

- In the bash terminal, `git --version` and press ENTER.
- The output should be `git version X` (where the X is the version number)
- *Don't worry if you don't have the exact same version as I do*

☑️ C) Do you have git configured?

- In the bash terminal, type `git config --list` and press ENTER
- You should see your name and email (and other things that aren't essential to configure)

☑️ D) Can you open a text editor? E.g.,

- Linux: gedit, nano
- macOS: textedit
- Windows: notepad

☑️ E) Can you go your GitHub account?

Linking git to GitHub

GitHub CLI (FYI; not recommended here)

- Lets you use GitHub from the terminal
- Makes workflows more efficient for software developers and maintainers
- Probably overkill for many of us

```
$ gh pr status

Current branch
    There is no pull request associated with [develop]

Created by you
    #1011  Update readme [readme-fix]
    - Checks pending - Review required

Requesting a code review from you
    #1015  Improve error handling [better-error-handling]
    ✓ Checks passing + Changes requested
```

Linking git to GitHub

SSH (FYI; not recommended here)

- Secure Shell Protocol (SSH Protocol)
- “a cryptographic network protocol for operating network services securely over an unsecured network.” (https://en.wikipedia.org/wiki/Secure_Shell)
- Often preferred by programmers
- May be more secure

- May be more challenging for those new to programming

Linking git to GitHub

Git Credit Manager (what we'll use today)

- Manages personal access token and 2-Factor Authentication

- **Windows**
  - If you installed Git for Windows, you already have it!

- **MacOS**
  - Type this in the terminal:
    `brew install --cask git-credential-manager`

- **Linux**
  - More complicated…
  - Install GCM
  - Configure git and GCM

Check if you're ready, con't

☑️ F) Have you connected git to your GitHub account?

- This will download the repo with the exercises.
  In the terminal, paste this:

  ```
  git clone https://github.com/ubc-library-rc/git-sandbox.git
  ```

Acknowledgements

Many parts of this presentation are inspired / based on these great resources:

- Chacon, S., & Straub, B. (2014). Pro git. Springer Nature. Available at https://git-scm.com/book/en/v2
- The Carpentries. (2021). Version Control with Git. https://swcarpentry.github.io/git-nov....
- https://docs.github.com/en/get-started/git-basics/set-up-git

Figure references

Here are the sources of some figures on my slides:

- The Carpentries. (2021). Version Control with Git. https://swcarpentry.github.io/git-nov....
- Chacon, S., & Straub, B. (2014). Pro git. Springer Nature. Available at https://git-scm.com/book/en/v2
- https://github.com/cli/cli#installation

The end

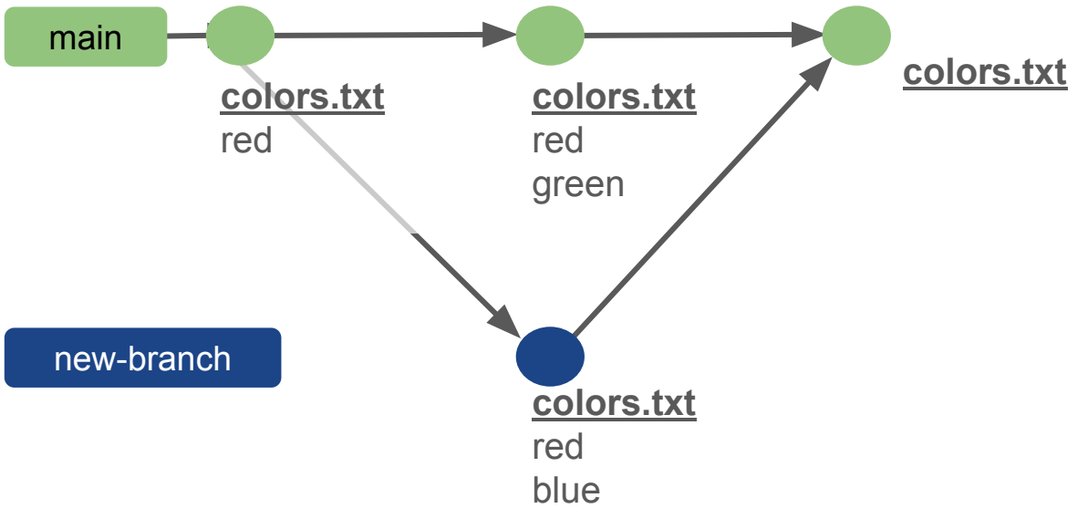
Options

- Commands can become powerful with options
  `git log --pretty=format:"%h - %an, %ar : %s" --graph`

- “How on earth will I remember that??”

- You can set aliases for commands you use a lot
  `git config --global alias.fancylog 'log --pretty=format:"%h - %an, %ar : %s" --graph'`

  `git fancylog`



What do you think you can do now?

(select as many as you want)

A) Explain why git and GitHub are useful
B) Track your own work with git
C) Share your own work on GitHub
D) Collaborate on GitHub

Basic Bash

- `cd <directory path>` changes directories (aka folders)
  - `.` represents the current directory
  - `..` represents the directory one level up

- `mkdir <directory path>` makes a new directory

- `touch <filename>` creates a new file

- `ls` lists the directory's contents
  - `ls -a` includes hidden files

- `<text editor name>` opens a text editor
  - `<text editor name> <filename>` opens a file in a text editor